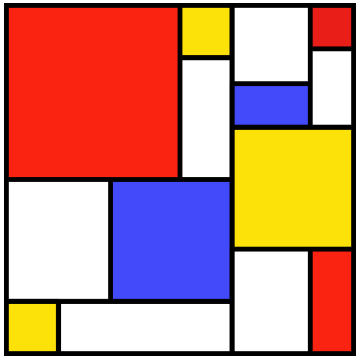
SLiM

Workshop Series



Callbacks I (`mutationEffect()`)

Benjamin C. Haller

Messer Lab, Cornell University

The WF Tick Cycle

The WF model's tick cycle

- 0. Execution of `first()` events
- 1. Execution of `early()` events
- 2. Generation of offspring:
  - 2.1. Choose source subpop
  - 2.2. Choose parent 1
  - 2.3. Choose parent 2 (`mateChoice()` callbacks)
  - 2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)
  - 2.5. Suppress/modify child (`modifyChild()` callbacks)
- 3. Removal of fixed mutations
- 4. Offspring become parents
- 5. Execution of `late()` events
- 6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks
- 7. Tick/cycle count increment

The WF model's tick cycle

0. Execution of `first()` events

1. Execution of `early()` events

2. Generation of offspring:
   - 2.1. Choose source subpop
   - 2.2. Choose parent 1
   - 2.3. Choose parent 2 (`mateChoice()` callbacks)
   - 2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)
   - 2.5. Suppress/modify child (`modifyChild()` callbacks)

3. Removal of fixed mutations

4. Offspring become parents

5. Execution of `late()` events

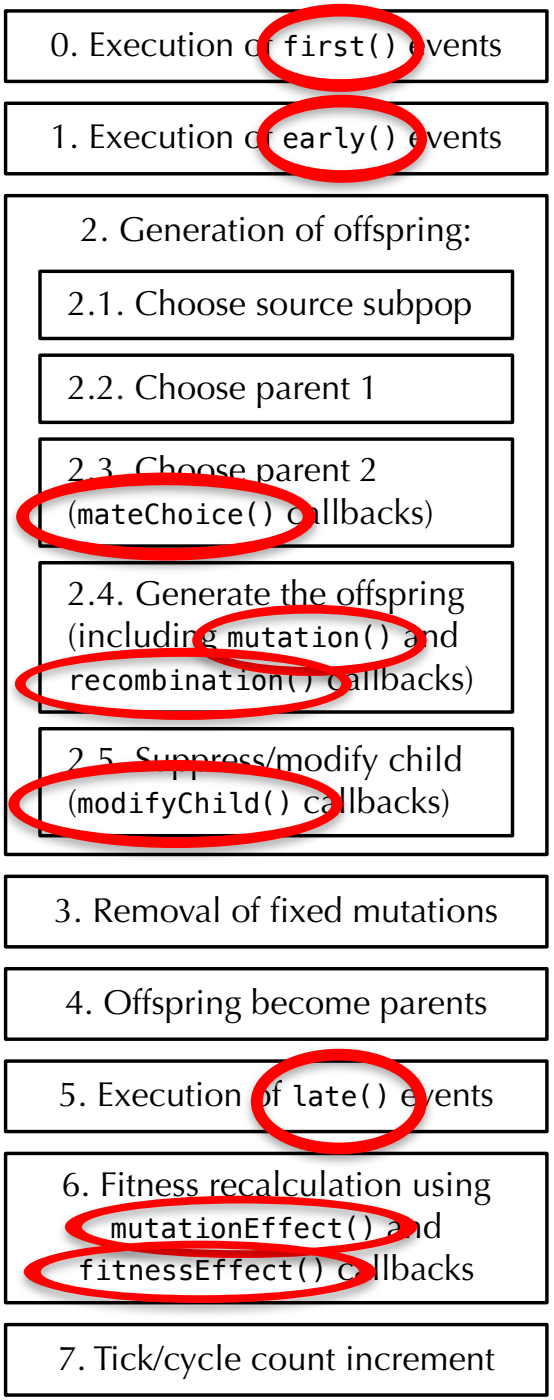
6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks

7. Tick/cycle count increment

The WF Tick Cycle

- a set of standard behaviors:
  - tick start
  - mate choice / reproduction
  - parental mortality
  - fitness value recalculation
  - tick end
- modified by *callbacks*
  - script providing new behavior

The WF model's tick cycle



The WF Tick Cycle

- Callbacks:
  - `first()` and `early()` events
  - `mateChoice()` callbacks
  - `mutation()` callbacks
  - `recombination()` callbacks
  - `modifyChild()` callbacks
  - `late()` events
  - `mutationEffect()` callbacks
  - `fitnessEffect()` callbacks
  - `interaction()`, `survival()`, `reproduction()`, ...

The WF model's tick cycle

0. Execution of `first()` events

1. Execution of `early()` events

2. Generation of offspring:

   2.1. Choose source subpop

   2.2. Choose parent 1

   2.3. Choose parent 2 (`mateChoice()` callbacks)

   2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)

   2.5. Suppress/modify child (`modifyChild()` callbacks)

3. Removal of fixed mutations

4. Offspring become parents

5. Execution of `late()` events

6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks

7. Tick/cycle count increment

`early()` events

```
early()
{
    ...
}
```

- called *early* in each tick
- often used to:
  - set up demographic parameters
  - prepare state for mating
  - make a plan for the tick

The WF model's tick cycle

0. Execution of `first()` events
1. Execution of `early()` events
2. Generation of offspring:
   - 2.1. Choose source subpop
   - 2.2. Choose parent 1
   - 2.3. Choose parent 2 (`mateChoice()` callbacks)
   - 2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)
   - 2.5. Suppress/modify child (`modifyChild()` callbacks)
3. Removal of fixed mutations
4. Offspring become parents
5. Execution of `late()` events
6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks
7. Tick/cycle count increment

`late()` events

```
late()
{
    ...
}
```

- called *late* in each tick
- often used to:
  - modify the offspring state
  - run behavioral interactions
  - prepare for fitness calculations
  - produce custom output

The WF model's tick cycle

0. Execution of `first()` events
1. Execution of `early()` events
2. Generation of offspring:
   - 2.1. Choose source subpop
   - 2.2. Choose parent 1
   - 2.3. Choose parent 2 (`mateChoice()` callbacks)
   - 2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)
   - 2.5. Suppress/modify child (`modifyChild()` callbacks)
3. Removal of fixed mutations
4. Offspring become parents
5. Execution of `late()` events
6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks
7. Tick/cycle count increment

`mutationEffect()`

- called during fitness calculation
- override the default effect of mutations on fitness
- often used to:
  - produce spatial variation
  - produce temporal variation
  - produce epistatic interactions
  - produce frequency dependence
  - make a mutation type neutral

The WF model's tick cycle

0. Execution of `first()` events
1. Execution of `early()` events
2. Generation of offspring:
   - 2.1. Choose source subpop
   - 2.2. Choose parent 1
   - 2.3. Choose parent 2 (`mateChoice()` callbacks)
   - 2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)
   - 2.5. Suppress/modify child (`modifyChild()` callbacks)
3. Removal of fixed mutations
4. Offspring become parents
5. Execution of `late()` events
6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks
7. Tick/cycle count increment

`mutationEffect()`

```
mutationEffect(<mutType> [, <subpop>])
{
    ...
}
```

- apply to *one* mutation type
- apply to *one or all* subpops
- called once for:
  - each **focal mutation**,
  - in each **focal individual**
  - allows individual effects
- return a **mutation effect**

The WF model's tick cycle

- 0. Execution of `first()` events
- 1. Execution of `early()` events
- 2. Generation of offspring:
  - 2.1. Choose source subpop
  - 2.2. Choose parent 1
  - 2.3. Choose parent 2 (`mateChoice()` callbacks)
  - 2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)
  - 2.5. Suppress/modify child (`modifyChild()` callbacks)
- 3. Removal of fixed mutations
- 4. Offspring become parents
- 5. Execution of `late()` events
- 6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks
- 7. Tick/cycle count increment

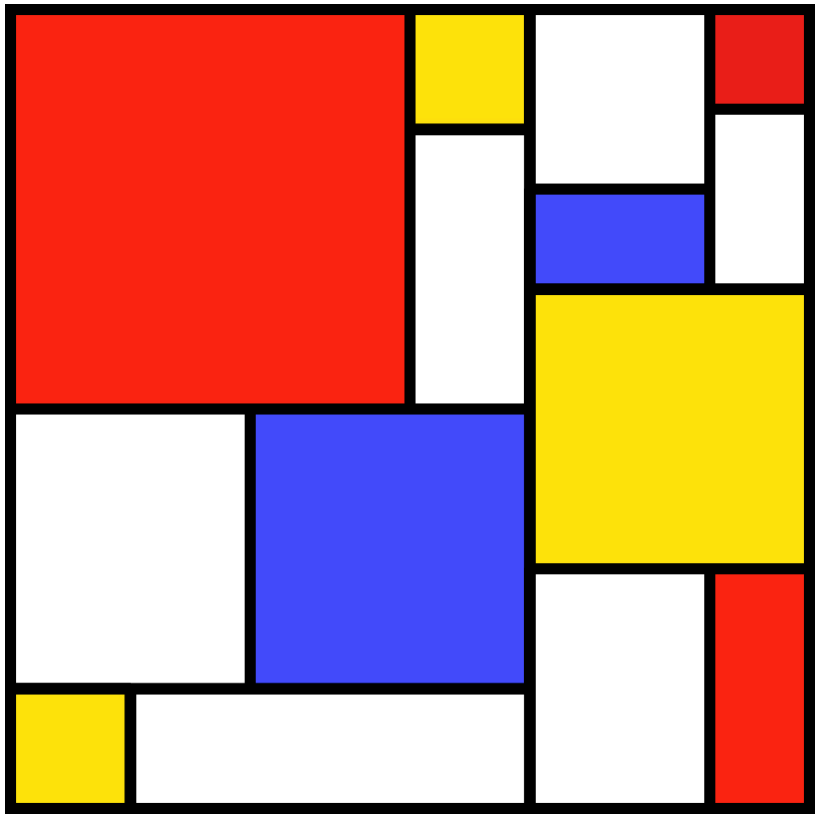
`mutationEffect()`

- there is a *focal mutation*
- it is in a *focal individual*
- how does the callback know?
- **pseudo-parameters**:
  - `mut`
  - `homozygous`
  - `individual`
  - `subpop`
  - `effect`

Fitness calculation

- Fitness values in SLiM:
  - evaluated **per individual**, **per mutation**
  - `homozygous`: `T` if homozygous, `F` if heterozygous
  - `effect` has the default mutation effect:

```
mutationEffect(m1)
{
   if (homozygous)
      return 1.0 + mut.selectionCoeff;
   else
      return 1.0 + mut.mutationType.dominanceCoeff *
                        mut.selectionCoeff;
}
```



Copyright © 2020–2025 Benjamin C. Haller. All Rights Reserved.